The energy crossover

Energy break-even analysis of parametric PINNs and classical PDE solvers

Course: Green Computing | Proposed duration: 12 weeks

1. Purpose and research question

A trained neural network may answer a physics problem quickly, but training consumes energy before the first useful prediction. This project will determine whether those initial costs are recovered through repeated use. We will compare the cumulative electricity consumption and runtime of a physics-informed neural network (PINN) with an efficient classical solver at matched accuracy.

The question is specific: for a defined family of heat-diffusion problems, how many new parameter queries are needed before the PINN uses less energy, and how does that threshold change with the required accuracy? An absent or impractically distant crossover will be reported as a result, not treated as a failed project.

2. Background and scope

Grossmann et al. (2024) found that PINNs did not outperform finite elements in solution time and accuracy on their tested PDEs, although some solution evaluations were faster. Huang et al. (2026) examine how setup costs and query volume affect solver choice. This proposal builds on that work by measuring setup and query energy under a stated workload and accuracy requirement.

A one-dimensional heat problem with a known solution will serve as the pilot. The main experiment will use steady diffusion on a two-dimensional square with two bounded parameters controlling a spatially varying conductivity. A small parametric PINN will take coordinates and those parameters as inputs. One query will return the solution on a fixed output grid for a previously unseen parameter setting. Geometry and boundary conditions will remain fixed. Parameter ranges and the evaluation protocol will be frozen after the pilot.

3. Comparison and measurement

The classical baseline will use finite differences with SciPy sparse solvers. Grid refinement and an exact or independently checked reference solution will establish accuracy. Meshes, sparsity patterns and any valid factorization or preconditioner reuse will be retained. Neither method will be charged for recomputing an identical cached answer. Both methods will be tested on the same held-out parameters and output grid, using relative L2 error targets of 1% and 0.1%; 0.01% is a stretch target within a fixed training budget.

We will record setup, training and query energy separately on one controlled machine. Available CPU and GPU energy counters will define a common measurement boundary; CodeCarbon will log readings or clearly identified estimates. After warm-up, long blocks of queries, synchronized GPU timing and repeated trials will reduce measurement noise. At least three training seeds will be used. Failed attempts and tuning costs will remain in the experiment log, with a stated retry policy for deployment-cost estimates.

4. Work plan and decision points

Figure 1 shows the twelve-week sequence. Full energy trials begin only after the pilot establishes usable measurements and both implementations pass the accuracy check for the target being tested.

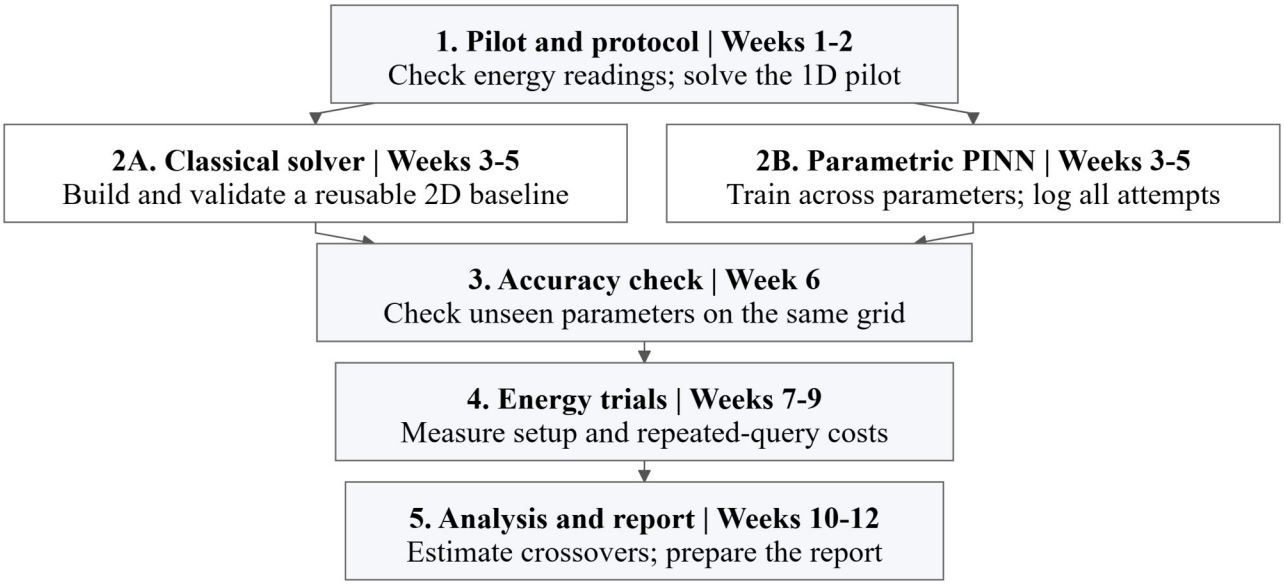


Figure 1. Project phases, with classical and neural implementations developed in parallel.

5. Analysis, resources and deliverables

For each method, cumulative energy will be estimated as follows; replacing energy with runtime gives the corresponding time model:

$$E_P(N) = S_P + N\ e_P \qquad ; \qquad E_C(N) = S_C + N\ e_C$$

Here, P denotes the PINN and C the classical solver; S includes all setup and training energy, e is energy per query and N is query count. Intersections define the energy and runtime break-even points. We will report uncertainty ranges, including cases with no reliable crossover. Runs that miss the accuracy target will be reported separately.

The work requires Python, PyTorch, SciPy and one machine with usable energy telemetry. The pilot will confirm hardware access and set a compute budget. Modeled power, if needed, will be labeled as an estimate and the measurement claim revised with the supervisor. Conclusions will remain specific to the tested hardware and parameter range.

The submission will include validated solver code, trained models or documented convergence failures, energy and runtime crossover plots, a comparison table with uncertainty, and reproducible run settings. A separate log will report the electricity used by the study itself. The intended outcome is a practical basis for choosing the lower-energy method for this workload.

References

Grossmann, T. G., Komorowska, U. J., Latz, J., & Schönlieb, C.-B. (2024). Can physics-informed neural networks beat the finite element method? IMA Journal of Applied Mathematics. https://doi.org/10.1093/imamat/hxae011

Huang, Y., Wang, X., Yao, S., Zeng, F., & Peng, W. (2026). Classical, machine learning, and hybrid solvers for partial differential equations: An evidence-weighted review and decision framework [Preprint]. arXiv. https://arxiv.org/abs/2610.32911